Panduan Website Day 15

Dari halaman produk biasa ke halaman yang interaktif

Isi panduan ini: **(1)** apa yang diminta Practical Guidance (PG), **(2)** apa yang kita kerjakan step by step, dan **(3)** cara minta bantuan AI dengan prompt yang jelas.

1. PG kita sebenarnya minta apa?

Fokus utama Day 15 adalah **dasar JavaScript**: tipe data dan operator (aritmatika, perbandingan, logika, assignment). Bagian website di PG berlabel **Overview**. Artinya kita melihat bagaimana dasar-dasar tadi dipakai di website sungguhan.

Peran JavaScript di website

Sisi	Tugas JavaScript
Front end	Mengatur interaksi pengguna, mengubah tampilan (DOM), dan berkomunikasi dengan API.
Back end (Node.js)	Menerima request, menjalankan logika bisnis, mengakses database, lalu mengirim response.

Target hasil setelah Day 15

#	Target	Contoh
1	Ada **2 halaman detail produk**: satu datanya hardcode di HTML, satu dari **API**	`product-1.html` dan `index.html?id=2`
2	Pengguna bisa **pilih varian dan jumlah**, lalu melihat **subtotal** tanpa reload	Footer bar: `Ukuran: M · Jumlah: 2 · Harga: Rp150.000`, **Subtotal Rp300.000**, tombol **Lanjutkan Pembayaran**
3	Kedua halaman memakai **id elemen yang sama**	`productForm`, `productPrice`, `variant`, `quantity`, `selectionSummary`
4	Paham **konversi input ke number** dengan `Number()` sebelum menghitung	`"150000" * "2"` vs `Number("150000") * Number("2")`
5	Data latihan dibedakan dari data asli API	Halaman API harus pakai harga dari database

Belum dikerjakan di Day 15: cek stok, pesan error, dan percabangan. Semua itu materi **Day 16** (if/else, switch, loop).

2. Apa yang kita kerjakan, step by step

Kenalan dulu dengan project-nya

Dalam satu project ada 3 halaman produk:

Halaman	Sumber data	Fungsi
`project-AI/product-1.html`	Hardcode di HTML	Product 1 untuk latihan

Halaman	Sumber data	Fungsi
`project-AI/index.html?id=2`	API `localhost:3000`	Product 2 untuk latihan (read saja)
`localhost:5173` (folder `frontend/`, Vite)	API yang sama	Halaman admin untuk **CRUD**: tambah, ubah, hapus produk

Alur datanya:

```
Admin Vite (5173) ──ubah data──► API (3000) ──► Database PostgreSQL
                                      │
index.html?id=2 ◄──baca data──────────┘
```

Kalau harga atau foto produk 2 diubah lewat admin Vite, setelah di-refresh `index.html?id=2` ikut berubah.

Kenapa port-nya beda (5500 dan 5173)?

Port	Dijalankan oleh	Dipakai untuk
5500	VS Code Live Server	HTML/CSS/JS biasa, langsung dibuka tanpa build (halaman pembeli)
5173	Vite dev server (`npm run dev`)	App yang ditulis dengan `import`/`export` (halaman admin)
3000	Node.js / Express	API yang dipakai oleh kedua halaman di atas

Di dunia nyata, halaman pembeli dan halaman admin memang biasanya dipisah, misalnya `tokobaju.com` dan `admin.tokobaju.com`. Karena alamatnya beda, API harus mengizinkan keduanya lewat **CORS**.

Step 1: Jalankan project-nya

1. Salin `.env.example` jadi `.env`.
2. Nyalakan database: `docker compose up -d`.
3. Siapkan tabel dan data contoh: `npm run migrate`, lalu `npm run seed`.
4. Jalankan API: `npm run dev` (cek `localhost:3000/health`).
5. Buka halaman dengan VS Code **Live Server** (`127.0.0.1:5500`).

Step 2: Perbaiki bug di kode lama

Sebelum menambah fitur, pastikan pondasinya benar.

Masalah	Perbaikan
Bug stok: stok 10 dengan 3 varian terbaca 30, karena stok disalin ke tiap varian lalu dijumlahkan	Stok disimpan satu kali di `products.stock` (migration `004`)
File Docker dobel: ada dua file compose yang bentrok	Disatukan ke `compose.yaml`
CORS: admin Vite belum diizinkan memanggil API	Alamat `localhost:5173` ditambahkan ke daftar origin

Step 3: Buat halaman Product 1 (hardcode)

`product-1.html` berisi **Kemeja Linen Biru**, Rp150.000, ukuran S/M/L, plus 3 foto. Semua data ditulis langsung di HTML, tanpa API.

Step 4: Tambahkan form pilihan di kedua halaman

```
<p id="productPrice" data-price="150000">Rp150.000</p>

<form id="productForm">
  <select id="variant">…</select>
  <input id="quantity" type="number" min="1" step="1" value="1" />
  <button type="submit">Update Pilihan</button>
</form>

<!-- Footer bar di bawah layar, muncul setelah Update Pilihan -->
<aside id="checkoutBar" hidden>
  <p id="selectionSummary"></p>   <!-- Ukuran: M · Jumlah: 2 · Harga: Rp150.000 -->
  <strong id="subtotalText"></strong>   <!-- Rp300.000 -->
  <button id="checkoutButton">Lanjutkan Pembayaran</button>
</aside>
```

`data-price` menyimpan harga sebagai **angka murni**, sedangkan teks `Rp150.000` cuma untuk tampilan. Ringkasan pilihan tampil di **footer bar** yang menempel di bawah layar, mirip halaman checkout di toko online.

Step 5: Tulis logika subtotal (product-interaction.js)

Inilah inti materi Day 15, yaitu tipe data dan operator:

```
const price = Number(priceElement.dataset.price);   // string → number
const quantity = Number(quantityInput.value);       // string → number
let subtotal = 0;
subtotal += price * quantity;                        // aritmatika + assignment
const hasSelection = variant !== '' && quantity > 0; // perbandingan + logika

summary.textContent = 'Ukuran: ' + variant + ' · Jumlah: ' + quantity
  + ' · Harga: Rp' + price.toLocaleString('id-ID');  // string + number
subtotalText.textContent = 'Rp' + subtotal.toLocaleString('id-ID');
checkoutBar.hidden = !hasSelection;                  // operator NOT, tanpa if/else
```

File ini dipakai oleh **semua** halaman produk. Perhatikan `checkoutBar.hidden = !hasSelection`: bar cuma muncul kalau pilihan lengkap, dan itu bisa dilakukan tanpa if/else. Tombol **Lanjutkan Pembayaran** untuk sekarang cuma menampilkan pesan demo.

Step 6: Sambungkan halaman Product 2 ke API

Di `script.js`, setelah data API datang:

- isi `data-price` dengan `product.price`,
- isi pilihan varian dari `product.variants.colorOrSize`,
- aktifkan tombol **Update Pilihan**. Sebelum data datang, tombolnya nonaktif supaya tidak menghitung dengan harga kosong.

Step 7: Parkir fitur lama untuk Day 16

Tombol +/− lama, Add to bag, dan batas stok di-comment dengan tanda `TODO(Day 16)`, karena butuh if/else.

Step 8: Uji hasilnya

Halaman	Input	Hasil yang benar
Semua halaman	belum klik Update Pilihan	Footer bar belum muncul

Halaman	Input	Hasil yang benar
Product 1	S × 2	Footer bar: `Ukuran: S · Jumlah: 2 · Harga: Rp150.000`, Subtotal **Rp300.000**
Product 2	varian apa saja × 2	Subtotal = **harga dari database × 2**
Semua halaman	klik Lanjutkan Pembayaran	Muncul pesan "Demo: halaman pembayaran belum tersedia."
Product 2 (API mati)	–	Tombol nonaktif, ada pesan "Pastikan API berjalan..."

Bonus: multiple image (contoh fitur end-to-end)

Satu fitur baru biasanya menyentuh **semua lapisan**:

Lapisan	Perubahan
Database	Migration `005` menambah kolom `images` (daftar URL foto)
API	Menerima dan mengirim `images`. Field lama `image` tetap ada, jadi kode lama nggak rusak
Admin Vite	Form punya input link foto (maksimal 3) dan tombol **+ Tambah foto**
Halaman produk	Menampilkan thumbnail galeri
Test	Test lama tetap lulus, dan ada test baru untuk fitur ini

3. Cara minta bantuan AI: contoh prompt yang bagus

Rumus prompt

Bagian	Isinya
Peran	AI berperan sebagai apa
Konteks	Project-nya apa, file mana yang terlibat, kondisi sekarang
Tugas	Apa yang harus diubah atau dibuat, **satu hal per prompt**
Batasan	Apa yang **tidak boleh** diubah, teknologi yang dipakai, batas materi
Format hasil	Mau dalam bentuk apa: kode lengkap, daftar file, penjelasan, cara tes

Tips

- Satu prompt untuk satu perubahan. Setelah dicek, baru lanjut ke prompt berikutnya.
- Sebutkan nama file dan id elemen dengan jelas.
- Minta AI **menjelaskan** kodenya dan **cara mengetesnya**.
- Selalu jalankan dan cek sendiri hasilnya. AI bisa salah.
- Jangan pernah tempel password atau isi file `.env` ke AI.

Urutan prompt dari awal sampai akhir

Prompt	Tujuan
A	Memahami project sebelum mengubah apa pun
B	Memperbaiki bug kode lama (stok)
C	Membuat halaman Product 1 (hardcode)

Prompt	Tujuan
D	Menambahkan form dan logika subtotal
E	Menyambungkan form ke data API (Product 2)
F	Minta AI mereview hasilnya
G	Menambah fitur baru end-to-end (multiple image)
H	Membuat checklist validasi akhir

Prompt A: Pahami project dulu

```
Kamu adalah mentor software engineer yang sabar.
Aku punya project dengan dua folder: api-product/ (Express + PostgreSQL)
dan project-AI/ (HTML/CSS/JS biasa + aplikasi Vite di frontend/).
Berikut struktur foldernya: [tempel hasil tree / daftar file].

Tolong jelaskan dengan bahasa sederhana:
1. Fungsi tiap folder dan file utama.
2. Bagaimana data mengalir dari database sampai tampil di halaman.
3. Kenapa ada 3 halaman produk dan bedanya apa.
Jangan ubah kode apa pun dulu.
```

Prompt B: Perbaiki bug kode lama

```
Kamu adalah backend developer Node.js + PostgreSQL.
Konteks: di api-product, produk dengan stok 10 dan 3 varian terbaca stok 30.
File terkait: migrations/002_create_product_variants.sql,
src/repositories/product.repository.js, src/utils/product-response.js.

Tugas: cari penyebab bug ini dan perbaiki.
Batasan:
- Bentuk response API tidak boleh berubah (tetap variants.quantity).
- Data lama jangan sampai hilang. Buat migration baru, jangan edit migration lama.
Hasil: jelaskan penyebabnya, tampilkan kode yang diubah per file,
lalu beri langkah untuk mengetes perbaikannya.
```

Prompt C: Buat halaman hardcode (Product 1)

```
Kamu adalah front-end developer HTML/CSS/JavaScript dasar.
Konteks: project-AI/index.html adalah halaman detail produk yang datanya dari API.

Tugas: buat project-AI/product-1.html dengan tampilan yang sama persis
(header, footer, galeri, styles.css), tapi semua data ditulis langsung di HTML:
- Nama: Kemeja Linen Biru, harga 150000, stok 25, ukuran S/M/L, 3 foto.
Batasan: jangan panggil API dan jangan pakai script.js. Jangan ubah styles.css.
Hasil: file HTML lengkap + penjelasan singkat bagian yang berbeda dari index.html.
```

Prompt D: Tambahkan form dan logika subtotal

```
Kamu adalah mentor JavaScript untuk pemula (materi: tipe data dan operator).
Konteks: ada dua halaman, product-1.html (hardcode) dan index.html (dari API).

Tugas:
1. Di kedua halaman, tambahkan <p id="productPrice" data-price="...">,
   <form id="productForm"> berisi select#variant, input#quantity (min 1, step 1),
   tombol "Update Pilihan". Di bawah layar, tambahkan footer bar <aside id="checkoutBar"
hidden>
   berisi <p id="selectionSummary">, <strong id="subtotalText">, dan tombol
   "Lanjutkan Pembayaran" (#checkoutButton).
2. Buat product-interaction.js yang dipakai kedua halaman:
   - ambil harga dan jumlah, ubah ke number dengan Number(),
   - hitung subtotal dengan subtotal += price * quantity,
   - hasSelection = variant !== '' && quantity > 0,
   - di footer bar tampilkan "Ukuran: M · Jumlah: 2 · Harga: Rp150.000"
     dan subtotal "Rp300.000" (pakai toLocaleString('id-ID')),
   - munculkan bar dengan checkoutBar.hidden = !hasSelection,
   - tombol "Lanjutkan Pembayaran" cukup menampilkan pesan demo.
Batasan: jangan pakai if/else, loop, function buatan sendiri, atau array.
Itu materi hari berikutnya. Jangan hapus fitur lama, cukup comment dengan TODO(Day 16).
Hasil: kode per file + penjelasan tiap baris product-interaction.js.
```

Prompt E: Sambungkan form ke data API (Product 2)

```
Konteks: index.html memakai script.js yang mengambil produk dari
GET http://localhost:3000/api/products/:id. Form #productForm sudah ada.

Tugas: setelah data API berhasil diambil, isi data-price pada #productPrice
dengan product.price, isi pilihan #variant dari product.variants.colorOrSize,
dan aktifkan tombol submit. Sebelum data datang, tombol harus nonaktif.
Batasan: jangan ubah product-interaction.js, dan jangan bikin nama variabel
global yang sama dengan yang ada di product-interaction.js.
Hasil: bagian script.js yang diubah + cara mengetes saat API hidup dan mati.
```

Prompt F: Minta AI mereview hasilnya

```
Kamu adalah code reviewer. Berikut kode product-1.html, index.html,
script.js, dan product-interaction.js: [tempel kode].

Review berdasarkan checklist ini:
- Id elemen sama di kedua halaman?
- Harga dan jumlah sudah dikonversi dengan Number() sebelum dihitung?
- Halaman API memakai harga dari API, bukan harga contoh?
- Ada error di console?
Jangan menulis ulang semuanya. Sebutkan masalahnya, lokasinya,
dan saran perbaikan singkat.
```

Prompt G: Fitur baru end-to-end (multiple image)

```
Kamu adalah full-stack developer (Express, PostgreSQL, Zod, Vite).
Konteks: sekarang satu produk cuma punya satu foto
(kolom image_url, field API "image").

Tugas: buat satu produk bisa punya sampai 3 foto.
1. Database: migration baru yang menambah kolom images (daftar URL),
   data lama tetap aman.
2. API: POST/PUT menerima "images" (opsional, URL http/https), response
   mengirim "images". Field "image" tetap ada sebagai foto utama.
3. Admin Vite: di form tambah/ubah, input link foto satu per satu,
   foto 1 = foto utama (wajib), tombol "+ Tambah foto" untuk input berikutnya
   (maksimal 3), dan tombol × untuk menghapus input.
4. Halaman detail: tampilkan thumbnail kalau fotonya lebih dari satu.
Batasan: jangan merusak kontrak API lama. Semua test lama harus tetap lulus.
Hasil: kerjakan per lapisan (database → API → UI). Untuk tiap lapisan,
tampilkan file yang diubah, test yang ditambahkan, dan cara mengeceknya.
```

Prompt H: Validasi akhir

```
Aku sudah menerapkan semua perubahan. Buatkan checklist pengujian manual
untuk: product-1.html, index.html?id=2, dan admin Vite (tambah/ubah foto).
Untuk tiap langkah, tulis: aksi yang dilakukan, hasil yang seharusnya,
dan apa yang dicek kalau hasilnya beda.
```

Ringkasan

Day 15: pakai tipe data dan operator supaya halaman produk bisa menghitung subtotal, baik dari data hardcode maupun dari API.

Day 16: tambahkan aturan (stok, validasi) dengan if/else, switch, dan loop.